

Todo/Task 事项管理系统深度分析

> 对 Claude Code CLI 项目中 Todo/Task 管理系统的全面剖析
> 调研日期: 2026-07-07

目录

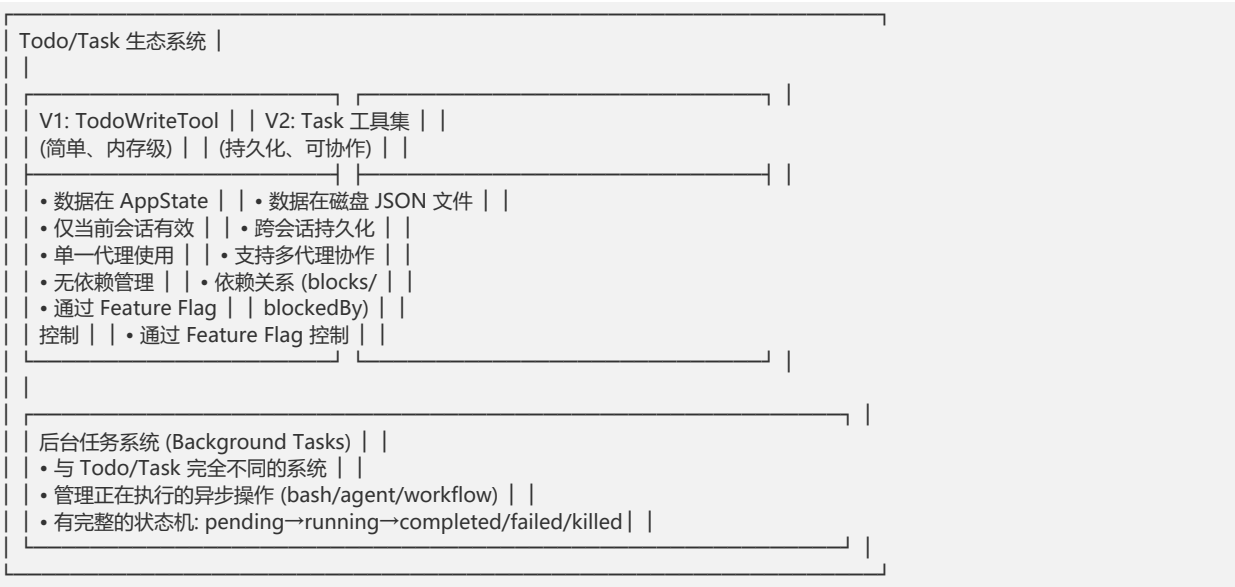
- 1. 整体架构概览
- 2. 两个并存的系统：Todo V1 与 Task V2
- 3. TodoWriteTool (V1) 详解
- 4. Task 工具集 (V2) 详解
- 5. 子智能体 (Sub-agent) 的 Task 工具访问权限
- 6. 任务生命周期管理
- 7. 后台任务系统 (Background Tasks)
- 8. 状态管理与 UI 渲染
- 9. 文件持久化机制
- 10. 动手实践：如何使用
- 11. 关键文件索引

1. 整体架构概览

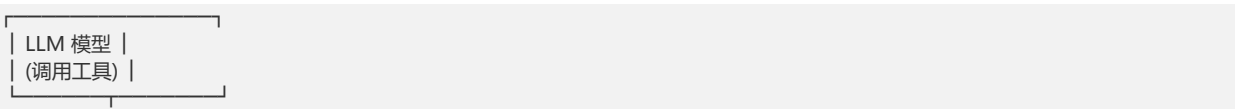
为什么需要两套 Todo/Task 系统？

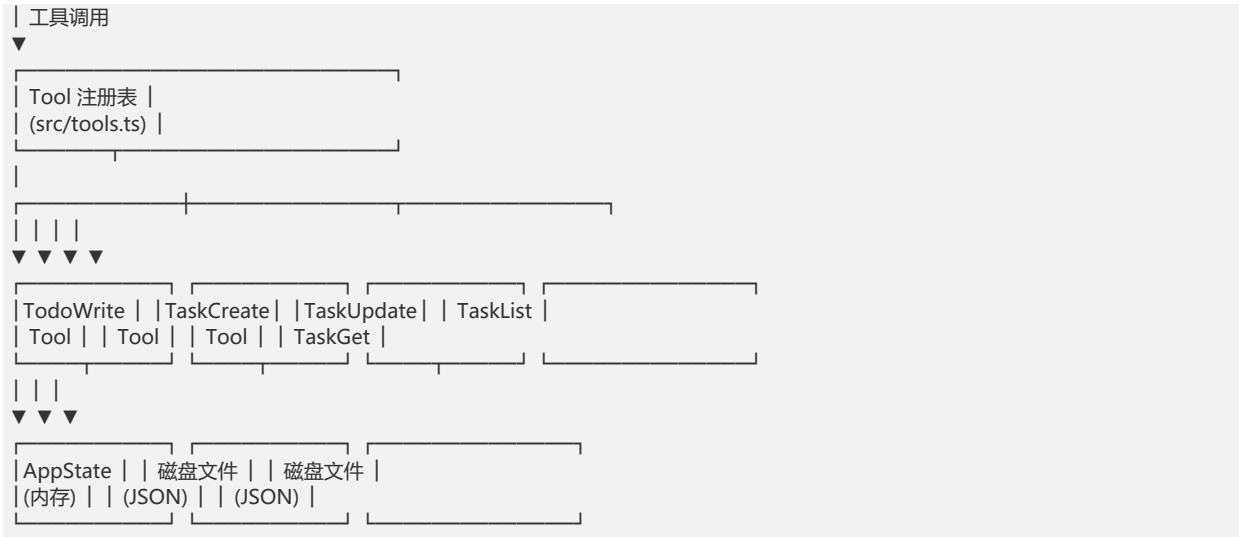
Claude Code 中存在 两套并行的任务管理系统，这是演进的结果：

- V1 (TodoWriteTool): 最初的简单方案，数据存在内存 (AppState) 中，仅用于当前会话
- V2 (TaskCreate/Update/List/GetTool): 后来引入的升级方案，数据存在磁盘上，支持跨会话持久化、团队协作、任务依赖



核心架构图





2. 两个并存的系统：Todo V1 与 Task V2

为什么有两个系统？

通过运行时开关 `isTodoV2Enabled()` 控制：

```
// src/utils/tasks.ts:133
export function isTodoV2Enabled(): boolean {
  if (isEnvTruthy(process.env.CLAUDE_CODE_ENABLE_TASKS)) {
    return true // 环境变量强制启用 V2
  }
  return !getIsNonInteractiveSession() // 非交互模式禁用 V2 (使用 V1)
}
```

- 交互式 CLI 模式 → V2 (任务工具集)
- 非交互/SDK 模式 → V1 (TodoWriteTool)
- 环境变量 `CLAUDECODEENABLE_TASKS` → 强制 V2

数据模型对比

维度	V1 (TodoWriteTool)	V2 (Task 工具集)
数据位置	`AppState.todos` (内存)	磁盘 JSON 文件
持久化	否，会话结束消失	是，存储在 `~/claude/tasks/`
字段	content, status, activeForm	id, subject, description, status, activeForm, owner, blocks, blockedBy, metadata
状态	pending/in_progress/completed	pending/in_progress/completed (+ deleted 为特殊操作)
依赖管理	无	blocks / blockedBy
所有者	无	owner (agent 名称)
协作	不支持	支持多 agent 认领
ID 生成	无 (数组索引)	自动递增数字 ID

3. TodoWriteTool (V1) 详解

怎么触发的？

LLM 模型在合适的场景下主动调用 TodoWrite 工具。触发时机由其 prompt 定义：

文件: src/tools/TodoWriteTool/prompt.ts

触发场景（prompt 中定义的 use cases）：

1. 复杂多步骤任务 — 3 个或以上步骤
2. 非平凡任务 — 需要规划或多个操作
3. 用户明确要求 — 用户说"用 todo list"
4. 用户列出了多个事项 — 用户说了多个任务
5. 收到新指令后 — 立即用 todo 记录需求
6. 开始工作时 — 标记为 in_progress
7. 完成任务后 — 标记完成并添加后续任务

不触发的场景：

- 单一简单任务（1 步完成）
- 纯咨询/信息性问题
- 单文件修改

怎么维护的？

```
// src/tools/TodoWriteTool/TodoWriteTool.ts:65
async call({ todos }, context) {
  const appState = context.getAppState()
  const todoKey = context.agentId ?? getSessionId()
  const oldTodos = appState.todos[todoKey] ?? []
  const allDone = todos.every(_ => _.status === 'completed')
  const newTodos = allDone ? [] : todos

  context.setAppState(prev => ({
    ...prev,
    todos: {
      ...prev.todos,
      [todoKey]: newTodos, // 按 agentId 或 sessionId 隔离
    },
  )))
}
```

- 每次调用时全量替换 todo 列表（不是增量更新）
- 按 agentId 或 sessionId 隔离不同的上下文
- 所有项完成时自动清空列表
- 数据存放在 AppState.todos 中

怎么添加提示词的？

```
// src/tools/TodoWriteTool/prompt.ts:3-184
export const PROMPT = `Use this tool to create and manage a structured task list...`

export const DESCRIPTION =
  'Update the todo list for the current session...'
```

Prompt 通过场景示例 + 反例教导模型何时使用：

- 4 个正面示例（何时使用）
- 4 个反面示例（何时不使用）
- 状态管理规则
- 完成标准

验证提示（重要特性）

当关闭 3+ 个任务但没有验证步骤时，V1 会发出验证提示。该功能有 6 个门控条件：

```
// TodoWriteTool.ts:77-86
if (
  feature('VERIFICATION_AGENT') && // ① Feature Flag: VERIFICATION_AGENT 已启用
```

```
getFeatureValue_CACHED_MAY_BE_STALE('tengu_hive_evidence', false) && // ② 运行时开关: tengu_hive_evidence 已开启
!context.agentId && // ③ 仅主线程 (非子 agent)
allDone && // ④ 全部任务已完成
todos.length >= 3 && // ⑤ 至少 3 个任务
!todos.some(t => /verif/i.test(t.content)) // ⑥ 没有验证步骤
){
  verificationNudgeNeeded = true
  // → 提示: 需要调用 VERIFICATION_AGENT 进行验证
}
```

注意：条件 ①-③ 容易被忽略。条件 ③ 意味着子 agent 关闭 todo 列表时不会触发验证提示。

4. Task 工具集 (V2) 详解

4.1 TaskCreateTool — 创建任务

文件: src/tools/TaskCreateTool/TaskCreateTool.ts

输入参数：

```
subject: string // 任务标题 (必须)
description: string // 详细描述 (必须)
activeForm?: string // 进行时态, 显示在 spinner (可选)
metadata?: object // 任意元数据 (可选)
```

执行流程：

```
用户/模型调用 TaskCreate
|
▼
生成自增 ID (从磁盘读取最高 ID + 1)
|
▼
创建 JSON 文件 → ~/.claude/tasks/<taskListId>/<id>.json
|
▼
执行 TaskCreated hooks (可阻断创建)
|
▼
hooks 失败 → 删除任务文件并报错
hooks 成功 → 展开任务面板 UI
|
▼
返回 { task: { id, subject } }
```

关键实现细节：

```
// src/utils/tasks.ts:284
export async function createTask(taskListId: string, taskData: Omit<Task, 'id'>): Promise<string> {
  // 使用文件锁防止并发冲突
  release = await lockfile.lock(lockPath, LOCK_OPTIONS)
  const highestId = await findHighestTaskId(taskListId)
  const id = String(highestId + 1)
  const task: Task = { id, ...taskData }
  await writeFile(path, jsonStringify(task, null, 2))
  notifyTasksUpdated()
  return id
}
```

4.2 TaskUpdateTool — 更新任务

文件: src/tools/TaskUpdateTool/TaskUpdateTool.ts

核心功能：

TaskUpdate 参数
taskId: string → 要更新的任务 ID
subject?: string → 修改标题

```
| description?: → 修改描述 |
| activeForm?: → 修改进行时态 |
| status?: → pending/in_progress/ |
| completed/deleted |
| owner?: → 指定所有者 |
| addBlocks?: → 添加被此任务阻塞的任务 |
| addBlockedBy?: → 添加阻塞此任务的任務 |
| metadata?: → 合并/删除元数据 |
```

状态流转：

```
pending —————→ in_progress —————→ completed
|
|————— 任何状态 → deleted (删除)
```

智能特性：

1. 自动设置所有者 — 当 agent 将任务标记为 in_progress 时，自动填写 owner
2. 任务完成 hooks — 标记完成时执行 TaskCompleted hooks
3. 邮箱通知 — 所有权变更时通过 mailbox 通知新 owner
4. 验证提示 — 关闭 3+ 任务时提示需要验证步骤
5. 团队成员提醒 — 完成任务时提示队友查看 TaskList

4.3 TaskListTool — 列出任务

文件: src/tools/TaskListTool/TaskListTool.ts

- 读取指定 taskListId 目录下的所有 JSON 文件
- 过滤内部任务 (metadata._internal)
- 自动过滤已完成的 blockedBy 引用
- 返回格式: #1 [pending] 任务标题 (owner) [blocked by #2]

团队 workflow 提示：

1. 完成任务后调用 TaskList 找可用工作
2. 找 status=pending, 无 owner, empty blockedBy 的任务
3. 优先按 ID 顺序处理
4. 用 TaskUpdate 认领任务 (设置 owner)
5. 如果被阻塞，先解阻塞任务或通知组长

4.4 TaskGetTool — 获取单个任务

文件: src/tools/TaskGetTool/TaskGetTool.ts

- 按 ID 获取完整任务详情 (含 description、blocks、blockedBy, 但不含 owner/activeForm/metadata)
- 适合开始工作前获取完整上下文

4.5 工具提示词 (Prompt) 详解

V2 系统中的每个工具都配有 Description 和 Prompt, 用于教导 LLM 何时使用以及如何使用。这些提示词是 LLM 正确使用 Task 工具的关键。

4.5.1 TaskCreateTool 的提示词

文件: src/tools/TaskCreateTool/prompt.ts

Description (简短描述, 用于工具注册表) :

```
Create a new task in the task list
```

Prompt (完整行为指南, 注入 LLM 系统提示词) :

提示词包含以下关键部分：

1. 使用场景 (When to Use This Tool) :

- 复杂多步骤任务 — 3 个或以上步骤
- 不平凡复杂任务 — 需要规划或多个操作
- Plan mode 场景
- 用户明确要求使用 todo list
- 用户列出了多个事项（编号或逗号分隔）
- 收到新指令后立即用 TaskCreate 记录需求
- 开始工作时标记为 in_progress
- 完成任务后标记完成并添加后续任务

2. 不使用的场景（When NOT to Use）：

- 单一简单任务（1 步完成）
- 琐碎任务（无需组织管理）
- 可在 3 步内完成的简单任务
- 纯咨询/信息性任务

3. 任务字段说明：

- subject：祈使句形式的简短标题
- description：详细描述
- activeForm（可选）：进行时态，显示在 spinner 中

4. 使用技巧：

- 创建清晰、具体的任务标题
- 创建后用 TaskUpdate 设置依赖关系
- 先调用 TaskList 避免创建重复任务

5. 条件性内容（通过 isAgentSwarmsEnabled() 控制）：

- 启用多智能体协作时，额外提示 "description 需包含足够细节以便其他 agent 理解和完成"
- 提示 "新任务状态为 pending、无 owner，使用 TaskUpdate 设置 owner 来分配任务"

4.5.2 TaskUpdateTool 的提示词

文件: src/tools/TaskUpdateTool/prompt.ts

Description: Update a task in the task list

Prompt 包含：

1. 何时标记完成：明确列出了可标记完成的情形和禁止标记的情形
2. 删除任务：deleted 状态永久删除任务
3. 更新字段：status, subject, description, activeForm, owner, metadata, addBlocks, addBlockedBy
4. 状态工作流：pending → in_progress → completed, deleted 为特殊操作
5. 陈旧性警告：更新前先用 TaskGet 获取最新状态
6. JSON 示例：直接给出了 5 种典型用法的 JSON 代码示例

4.5.3 TaskGetTool 的提示词

文件: src/tools/TaskGetTool/prompt.ts

Description: Get a task by ID from the task list

Prompt 包含：

- 开始工作前获取完整上下文
- 查看任务依赖 (blocks/blockedBy)
- 被分配任务后获取完整需求
- 返回字段：subject, description, status, blocks, blockedBy

- 提示：取任务后先确认 blockedBy 为空再开始工作

4.5.4 TaskListTool 的提示词

文件: src/tools/TaskListTool/prompt.ts

Description: List all tasks in the task list

Prompt 包含：

- 查看可用任务 (status=pending, 无 owner, 未被阻塞)
- 检查整体进度
- 寻找被阻塞的任务
- 优先按 ID 顺序处理
- 返回字段：id, subject, status, owner, blockedBy

团队工作流 (isAgentSwarmsEnabled() 启用时额外注入)：

1. 完成任务后调用 TaskList 找可用工作
2. 找 status=pending, 无 owner, empty blockedBy 的任务
3. 优先按 ID 顺序处理
4. 用 TaskUpdate 认领任务 (设置 owner)
5. 如果被阻塞，先解阻塞任务或通知组长

4.6 子智能体 (Sub-agent) 对 Task 工具的访问权限

> 详细分析见 第 5 章。

简而言之：

- 同步（前台）子智能体 — 可以访问 Task 工具（不在禁止列表中）
- 异步（后台）子智能体 — 不能访问 Task 工具（不在白名单中）
- In-process teammate — 可以访问 Task 工具（显式列入白名单）
- 主智能体和子智能体共享同一个 task list（基于 session ID）

4.7 TaskCreate 的一次一任务 vs TodoWrite 的批量模式

> 完整讨论见 第 5.5 节。

特性	V1 TodoWriteTool	V2 TaskCreateTool
一次调用创建	批量（整个列表）	单个任务
更新方式	全量替换	增量更新（TaskUpdate）
调用次数	1 次 = 创建/更新整个列表	8 个任务 = 8 次 TaskCreate

5. 子智能体 (Sub-agent) 的 Task 工具访问权限

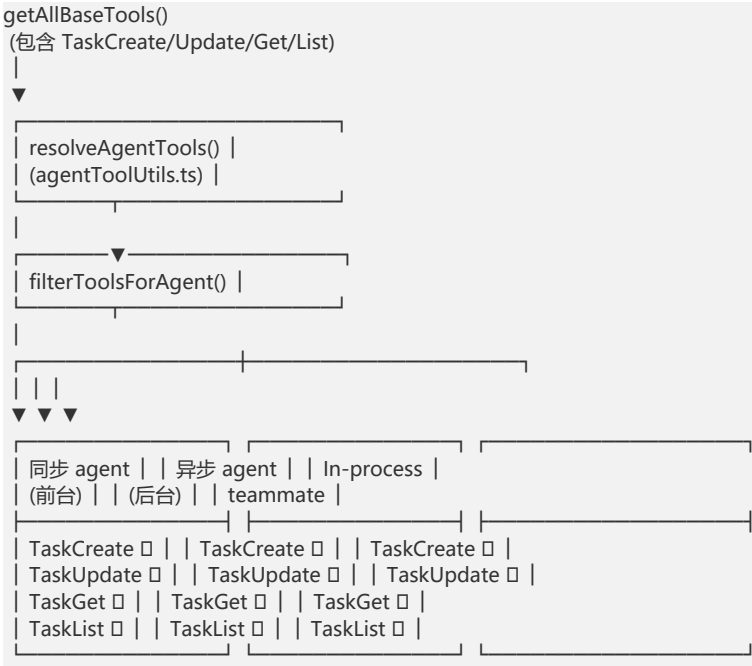
5.1 权限矩阵

子智能体能否访问 Task 工具，取决于其类型和运行方式：

子智能体类型	可访问 TaskCreate/Update/Get/List?	原因
同步（前台）子智能体（如 Explore、Plan、code-reviewer）	是	不在 `ALL_AGENT_DISALLOWED_TOOLS` 中
异步（后台）子智能体（`run_in_background=true`）	否	不在 `ASYNC_AGENT_ALLOWED_TOOLS` 中
In-process teammate（队友模式）	是	显式列入 `IN_PROCESS_TEAMMATE_ALLOWED_TOOLS`

子智能体类型	可访问 TaskCreate/Update/Get/List?	原因
主线程 (根) agent	是	直接使用完整工具池

5.2 过滤机制图解



5.3 关键代码

文件: src/constants/tools.ts

```
// 所有 agent 全局禁止的工具 (Task 工具不在其中 → 默认允许)
// AGENT_TOOL_NAME 仅非 ant 用户禁止; WORKFLOW_TOOL_NAME 有 feature flag 控制
export const ALL_AGENT_DISALLOWED_TOOLS = new Set([
  TASK_OUTPUT_TOOL_NAME, // 禁止
  EXIT_PLAN_MODE_V2_TOOL_NAME, // 禁止
  ENTER_PLAN_MODE_TOOL_NAME, // 禁止
  ...(process.env.USER_TYPE === 'ant' ? [] : [AGENT_TOOL_NAME]),
  ASK_USER_QUESTION_TOOL_NAME, // 禁止
  TASK_STOP_TOOL_NAME, // 禁止
  ...(feature('WORKFLOW_SCRIPTS') ? [WORKFLOW_TOOL_NAME] : []),
])

// 异步 agent 白名单 (Task 工具不在其中 → 禁止)
export const ASYNC_AGENT_ALLOWED_TOOLS = new Set([
  FILE_READ_TOOL_NAME, WEB_SEARCH_TOOL_NAME, TODO_WRITE_TOOL_NAME, ...
])

// In-process teammate 额外允许的工具
export const IN_PROCESS_TEAMMATE_ALLOWED_TOOLS = new Set([
  TASK_CREATE_TOOL_NAME, TASK_GET_TOOL_NAME,
  TASK_LIST_TOOL_NAME, TASK_UPDATE_TOOL_NAME,
  SEND_MESSAGE_TOOL_NAME,
])
```

文件: src/tools/AgentTool/agentToolUtils.ts (过滤逻辑) :

```
export function filterToolsForAgent({ tools, isBuiltIn, isAsync, permissionMode }) {
  return tools.filter(tool => {
    // MCP 工具对所有 agent 开放
    if (tool.name.startsWith('mcp_')) return true
    // Plan 模式下的 ExitPlanMode 绕过
    if (toolMatchesName(tool, EXIT_PLAN_MODE_V2_TOOL_NAME) && permissionMode === 'plan') return true
    if (ALL_AGENT_DISALLOWED_TOOLS.has(tool.name)) return false
  })
}
```



```
if (!isBuiltin && CUSTOM_AGENT_DISALLOWED_TOOLS.has(tool.name)) return false
if (isAsync && !ASYNC_AGENT_ALLOWED_TOOLS.has(tool.name)) {
  if (isAgentSwarmsEnabled() && isInProcessTeammate()) {
    if (IN_PROCESS_TEAMMATE_ALLOWED_TOOLS.has(tool.name)) return true
  }
  return false // 异步 agent 禁止
}
return true
})
}
```

5.4 共享同一个 Task List

主智能体和子智能体共享同一个 taskListId，因为 getTaskListId() 的 fallback 是 getSessionId()，而子智能体通过 createSubagentContext() 继承父级的 session：

```
// src/utils/tasks.ts:199
export function getTaskListId(): string {
  if (process.env.CLAUDE_CODE_TASK_LIST_ID) return ...
  const teammateCtx = getTeammateContext()
  if (teammateCtx) return teammateCtx.teamName // 队友共用 leader 的 task list
  return getTeamName() || leaderTeamName || getSessionId()
}
```

这意味着主智能体创建的任务，前台子智能体可以看到并更新，反之亦然。这是团队协作的基础。

5.5 "8 个步骤需要 8 次 TaskCreate" 问题

是的，需要 8 次调用。V2 的 TaskCreateTool 一次调用只能创建一个任务。这与 V1 的 TodoWriteTool 形成鲜明对比：

特性	V1 TodoWriteTool	V2 TaskCreateTool
一次调用创建	批量（整个列表）	单个任务
更新方式	全量替换	增量更新（TaskUpdate）
8 个步骤	1 次 TodoWrite 调用	8 次 TaskCreate 调用

优化策略：8 次 TaskCreate 可以并行发出（同一个 AI 回复中多个工具调用），因为它们没有数据依赖——每个 TaskCreate 通过文件锁独立分配 ID：

```
AI 回复中同时发出（并行）：
TaskCreate({subject: "步骤1", description: "..."})
TaskCreate({subject: "步骤2", description: "..."})
TaskCreate({subject: "步骤3", description: "..."})
...

完成后，再并行设置依赖：
TaskUpdate({taskId: "2", addBlockedBy: ["1"]})
TaskUpdate({taskId: "3", addBlockedBy: ["1"]})
```

6. 任务生命周期管理

V2 任务的完整生命周期

V2 任务的状态模型仅包括三种核心状态，加上一种特殊操作：





- 状态: pending → in_progress → completed
- deleted 不是独立状态，而是 TaskUpdate 工具的特殊操作——它将任务文件从磁盘删除
- V2 中没有 failed 或 blocked 状态（这些是后台任务系统的概念，见第 6 章）

文件锁与并发安全

由于 V2 系统支持多 agent 并发操作，使用了文件锁机制：

```
// src/utls/tasks.ts:102-108
const LOCK_OPTIONS = {
  retries: {
    retries: 30, // 最多重试 30 次
    minTimeout: 5, // 初始等待 5ms
    maxTimeout: 100, // 最大等待 100ms
  },
}
```

锁的粒度：

- 任务列表锁 (.lock 文件)：创建任务、重置列表时使用
- 任务文件锁 (<id>.json 文件)：更新、删除单任务时使用

High Water Mark 机制

为了防止删除任务后 ID 被重用（造成混淆），使用 high water mark 文件：

```
// src/utls/tasks.ts:92
// .highwatermark 文件存储最大 ID 值
// 删除任务时更新它，新建任务时从 max(文件ID, 水印ID) 取下一个

首次创建 删除任务 #3 创建新任务
| | |
▼ ▼ ▼
文件: 1.json 2.json 文件: 1.json 2.json 文件: 1.json 2.json 4.json
水印: 0 水印: 0 水印: 3 水印: 3
↑ ↑
删除时记录 新 ID = 水印 + 1 = 4
```

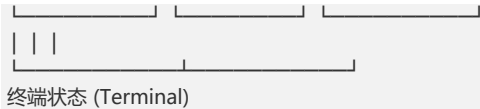
7. 后台任务系统（Background Tasks）

这是一个与 Todo/Task 不同的系统！

后台任务管理正在执行的操作（bash 命令、agent 调用），而非待办事项。

状态机





7 种任务类型

```
// src/Task.ts:6-13
export type TaskType =
  | 'local_bash' // 本地 bash 命令
  | 'local_agent' // 本地 agent 子进程
  | 'remote_agent' // 远程 agent
  | 'in_process_teammate' // 同进程队友
  | 'local_workflow' // 本地工作流
  | 'monitor_mcp' // MCP 监控
  | 'dream' // 梦境模式
```

轮询机制（事件驱动，非定时轮询）

重要纠正：pollTasks() 函数虽然定义了（src/utils/task/framework.ts:255），但从未被任何代码调用。真正被调用的是 generateTaskAttachments(), 它通过以下路径触发：

触发时机 1: 用户输入处理时
用户发送消息 → processUserInput.ts:504
↳ getAttachmentMessages() → getAttachments() → 主线程时调用
getUnifiedTaskAttachments() → generateTaskAttachments()

触发时机 2: 每个工具调用回合
queryLoop 工具循环 → query.ts:1580
↳ getAttachmentMessages() → getAttachments() → 主线程时调用
getUnifiedTaskAttachments() → generateTaskAttachments()

后台任务检查不是定时轮询，而是事件驱动的——每次工具循环迭代或用户输入时顺带检查一次。如果 AI 一次回复中调用了 5 个工具，就会检查 5 次；如果是纯聊天（无工具调用），则完全不触发。

用户输入 → processUserInput → 检查后台任务 (仅主线程)

|
| (AI 开始回复)

▼

queryLoop 工具循环

- └─ 工具调用 #1 → 工具循环 → 检查后台任务
- └─ 工具调用 #2 → 工具循环 → 检查后台任务
- └─ 工具调用 #3 → 工具循环 → 检查后台任务
- └─ ...

POLLINTERVALMS = 1000 常量定义在 framework.ts:22 但没有任何代码引用它——不仅 pollTasks() 未使用它，整个代码库都没有导入这个导出常量。它属于死代码。generateTaskAttachments() 执行的流程与报告的流程图一致，只是触发方式不同。

TOCTOU 安全防护

generateTaskAttachments 是异步的（读取磁盘），而任务状态可能在此期间变化。使用增量补丁模式防止：

```
// framework.ts:213
export function applyTaskOffsetsAndEvictions(
  setAppState: SetAppState,
  updatedTaskOffsets: Record<string, number>,
  evictedTaskIds: string[],
): void {
  setAppState(prev => {
    // 合并时再次检查最新状态
    for (const id of offsetIds) {
      const fresh = newTasks[id]
      if (fresh?.status === 'running') { // 只有 still running 才更新偏移量
        newTasks[id] = { ...fresh, outputOffset: updatedTaskOffsets[id]! }
      }
    }
    for (const id of evictedTaskIds) {
```

```
const fresh = newTasks[id]
if (!fresh || !isTerminalTaskStatus(fresh.status) || !fresh.notified) continue
delete newTasks[id]
}
})
}
```

输出磁盘写入

```
// src/utils/task/diskOutput.ts
export const MAX_TASK_OUTPUT_BYTES = 5 * 1024 * 1024 * 1024 // 5GB 上限
```

- 使用 O_NOFOLLOW 防止符号链接攻击 (Unix)
- 使用队列 + 单 drain 循环实现异步写入
- 超 5GB 后截断并写入 [output truncated: exceeded 5GB disk cap]
- 支持增量读取 (getTaskOutputDelta)，避免一次性加载大文件

关于 generateTaskAttachments 的补充说明

generateTaskAttachments() 虽然定义了 TaskAttachment 类型和 attachments 数组，但实际代码中该数组从未被填充——所有 completed 任务的通知由各任务类型自己的 enqueuePendingNotification() 回调处理，以避免重复通知。generateTaskAttachments() 的主要作用是计算 updatedTaskOffsets 和 evictedTaskIds，而非生成附件。

8. 状态管理与 UI 渲染

AppState 数据结构

```
// src/state/AppStateStore.ts:220
todos: { [agentId: string]: TodoList } // V1 todo 数据

// AppStateStore.ts:160
tasks: { [taskId: string]: TaskState } // 后台任务状态 (非 V2 todo)
```

React 订阅模式

```
// src/state/AppState.tsx
export function useAppState<T>(selector: (state: AppState) => T): T
export function useSetAppState(): (updater: (prev: AppState) => AppState) => void
```

- useAppState — 订阅特定状态片段，仅在选中值变化时重渲染
- useSetAppState — 获取更新函数但不订阅变化

后台任务面板

后台任务面板只显示运行中和待处理的任务（不显示已完成/失败的任务），通过 /tasks 命令打开：

```
| Background Tasks 面板 |
| |
| □ b8n3x... npm install [running] |
| □ a9m2k... code analysis [running] |
| □ d4f7p... build check [pending] |
| |
| [/tasks 命令打开此面板] |
```

注意：已完成 (completed) 和失败 (failed/killed) 的任务不会显示在此面板中——它们在达到终端状态后会被逐步驱逐出状态树。

通过 /tasks 命令打开：

```
// src/commands/tasks/tasks.tsx
export async function call(onDone, context) {
  return <BackgroundTasksDialog toolUseContext={context} onDone={onDone} />
}
```

9. 文件持久化机制

V2 任务的存储位置

```
~/claude/tasks/
├── <taskId> / ← 任务列表目录
│   ├── .lock ← 文件锁
│   ├── .highwatermark ← 最高 ID 记录
│   ├── 1.json ← 任务 #1
│   ├── 2.json ← 任务 #2
│   └── ...
└── ...
```

taskId 的解析优先级：

1. CLAUDE_CODE_TASK_LIST_ID 环境变量
2. 团队成员上下文中的 teamName
3. getTeamName() (动态团队上下文)
4. leaderTeamName (内存变量)
5. Session ID (回退)

注意：虽然代码注释中提到了 CLAUDECODETEAM_NAME，但实际代码并未读取该环境变量。getTeamName() 检查的是进程内队友上下文和动态团队上下文，而非环境变量。

后台任务的输出文件

```
<project_temp>/<sessionId>/tasks/
├── b8n3x123.output ← bash 任务输出
├── a9m2x456.output ← agent 任务输出
└── ...
```

- 使用 O_EXCL 防止文件已存在时被覆盖
- 使用 O_NOFOLLOW 防止符号链接攻击
- 使用 session ID 隔离不同会话

10. 动手实践：如何使用

V1 TodoWriteTool 使用方式

(模型在合适场景自动调用，用户不直接使用)

```
// LLM 内部调用：
TodoWrite({
  todos: [
    { content: "分析代码库结构", status: "in_progress", activeForm: "分析代码库结构" },
    { content: "编写测试用例", status: "pending", activeForm: "编写测试用例" },
    { content: "运行构建检查", status: "pending", activeForm: "运行构建检查" },
  ]
})
```

V2 Task 工具集使用方式

```
// 1. 创建任务
TaskCreate({
  subject: "实现用户登录功能",
  description: "需要创建登录表单、API 接口、JWT 验证",
  activeForm: "实现用户登录功能"
})
// → 返回 { task: { id: "1", subject: "..." } }

// 2. 创建更多任务并建立依赖
TaskCreate({ subject: "编写单元测试", description: "...", activeForm: "..." })
// → 返回 { task: { id: "2", subject: "..." } }
```

```
TaskUpdate({ taskId: "2", addBlockedBy: ["1"] })
// → task #2 被 task #1 阻塞

// 3. 开始工作
TaskUpdate({ taskId: "1", status: "in_progress" })
// → 自动设置 owner

// 4. 列出所有任务
TaskList()
// → #1 [in_progress] 实现用户登录功能 (agent-name)
// → #2 [pending] 编写单元测试 [blocked by #1]

// 5. 获取任务详情
TaskGet({ taskId: "1" })
// → 完整任务信息

// 6. 完成任务
TaskUpdate({ taskId: "1", status: "completed" })

// 7. 删除任务
TaskUpdate({ taskId: "2", status: "deleted" })
```

11. 关键文件索引

文件	用途	重要性
`src/tools/ToDoWriteTool/ToDoWriteTool.ts`	V1 ToDo 写入工具	□□□
`src/tools/ToDoWriteTool/prompt.ts`	V1 触发提示词	□□□
`src/utils/todo/types.ts`	ToDoItem/ToDoList 类型定义	□□
`src/tools/TaskCreateTool/TaskCreateTool.ts`	V2 任务创建工具	□□□
`src/tools/TaskCreateTool/prompt.ts`	V2 创建触发提示词	□□□
`src/tools/TaskUpdateTool/TaskUpdateTool.ts`	V2 任务更新工具	□□□
`src/tools/TaskUpdateTool/prompt.ts`	V2 更新触发提示词	□□□
`src/tools/TaskListTool/TaskListTool.ts`	V2 任务列工具	□□□
`src/tools/TaskListTool/prompt.ts`	V2 列表触发提示词	□□
`src/tools/TaskGetTool/TaskGetTool.ts`	V2 任务获取工具	□□
`src/tools/TaskGetTool/prompt.ts`	V2 获取触发提示词	□□
`src/utils/tasks.ts`	任务 CRUD 核心 + 文件锁	□□□□□
`src/Task.ts`	后台任务类型定义	□□□□
`src/tasks.ts`	任务类型注册表	□□
`src/utils/task/framework.ts`	后台任务状态管理 (generateTaskAttachments/applyTaskOffsets, pollTasks 未使用)	□□□□□
`src/utils/task/diskOutput.ts`	后台任务磁盘输出	□□□□
`src/state/AppStateStore.ts`	状态定义 (todos/tasks 字段)	□□□
`src/state/AppState.tsx`	React 状态 Hooks	□□
`src/tools/TaskStopTool/TaskStopTools`	后台任务停止工具	□□□
`src/tools/TaskOutputTool/TaskOutputTool.tsx`	后台任务输出读取	□□
`src/commands/tasks/tasks.tsx`	`/tasks` 命令 (面板 UI)	□□

文件	用途	重要性
`src/tasks/types.ts`	TaskState 类型	□□

总结

方面	V1 (TodoWriteTool)	V2 (Task 工具集)	后台任务系统
用途	待办清单	结构化任务管理	运行中操作监控
触发方式	LLM 自主调用	LLM 自主调用	工具循环迭代/用户输入时顺带检查
数据存储	内存 (AppState)	磁盘 JSON 文件	内存 + 磁盘输出
持久化	否	是	输出持久化
并发安全	单线程	文件锁	增量补丁
生命周期	单会话	跨会话	进程级
Feature Flag	`!isTodoV2Enabled()`	`isTodoV2Enabled()`	始终启用